

M H V A N



AI Context Exporter

Official Documentation

AI Context Exporter – Official Documentation

Overview & Philosophy

AI Context Exporter (Asset Context Exporter For Ai) is a C++ Editor plugin designed to translate your Unreal Engine project's binary files (.uasset, .umap) into highly optimized, machine readable text formats (JSONL). It serves as a bridge between Unreal Engine and Large Language Models (LLMs) or Agentic AI, allowing developers to generate structured context from Blueprints, Actor Components, and project hierarchies.

Context Exporter approaches AI context generation with a focus on privacy, portability, and zero friction:

- **Absolute Privacy & Local Scoping:** The tool operates entirely offline. Your project data remains strictly on your local hard drive and never passes through external servers or APIs during extraction. The AI only sees the specific scopes you explicitly choose to export and upload.
- **Zero Friction:** No Python environments to configure, no external dependencies, and no command-line setups. Install the plugin, run the export, and attach the resulting files to the LLM of your choice (ChatGPT, Claude, Gemini, etc.).
- **Portability:** Because the output is standard .jsonl, you can use your project's architecture for a variety of purposes: drag-and-drop context windows, building a local Vector Database (RAG pipeline), or fine-tuning offline models.

Quick Start Guide

1. Launch your Unreal Engine project.
2. On the top menu bar, click on Tools > Asset Context Exporter For AI.
3. In the UI, choose a Preset (e.g., *Core Architecture* or *Full Project*).
4. Under Export Settings, choose your preferred format (e.g., *Monolithic JSONL* or *File Splitting*).
5. Choose an output directory to store all the exported files.
6. Click RUN CONTEXT EXPORT.
7. Once finished, click Open Export Folder to retrieve your .jsonl files. Attach these to your LLM prompt.

Core Features & The Extraction Engine

The core of the tool runs on a dedicated, non-blocking Commandlet (UAIContextExporterCommandlet), allowing the editor to remain responsive during extensive exports. It iterates through the FAssetRegistryModule to extract data via Unreal's reflection system.

- **AST & Logic Translation:** Parses uncompiled visual scripting by calling functions like BuildDeepGraphAST() and BuildAnimGraphAST(). It translates UEdGraphNode execution pins, pure math logic, and variable sets into structured JSON.
- **Component Architecture Extraction (ECS):** By reading both the Class Default Object (CDO) and the Simple Construction Script (SCS_Node), the tool outputs the exact spatial and hierarchical relationships of USceneComponent and UChildActorComponent templates.
- **Delta Serialization (SerializeProperty):** Evaluates property values against their native CDO defaults. It only serializes variables explicitly modified by the developer, significantly reducing the JSON payload size and removing massive amounts of redundant Engine-default data.
- **Path Compression:** Implements an internal aliasing system that substitutes deep package paths (/Game/...) with compressed integer identifiers ([#ID]), optimizing LLM token usage.
- **Multilingual Support:** The extraction engine natively supports 8 languages:
 1. English (en) (*Native*).
 2. Japanese (ja).
 3. Simplified Chinese (zh-Hans).
 4. Korean (ko).
 5. German (de).
 6. Spanish (es).
 7. French (fr).
 8. Brazilian Portuguese (pt-BR).

The plugin interface seamlessly adapts to the active language of the Unreal Editor.

Important Note on Exported Data: To ensure strict compatibility and maintain structural integrity for AI parsing, only the plugin's User Interface is translated. The generated export folder tree (e.g., OO_Project_Manifest, 01_AI_Insights_And_Audits, 02_Architecture_Graph, etc.) is strictly hardcoded to export in English (en). The language of the files and text inside the exported files will depend entirely on the original content authored within your project's assets, though it will predominantly be structured in English (en) since most LLMs are significantly more effective at processing and reasoning in said language.

Targeted Scopes

You have total control over the data payload. Using the UI, toggle extraction modules based on your current AI task:

- **Source Code (.cpp / .h):** Extracts raw C++ implementation and Engine .ini configuration files.
- **Local Plugins:** Capability to include local third-party plugin source code (requires user FAB/EULA compliance verification).
- **Blueprints (Logic & Nodes):** Parses uncompiled Blueprint logic, properties, execution links, and component structures.
- **World Levels (Actors & Partitions):** Exports Actor transforms, localized properties, and tags inside .umap environments.
- **VFX & Materials:** Captures Niagara Systems and Material Instances.
- **Animations:** Extracts Animation Blueprints, Sequences, and Montages.
- **AI Logic:** Parses Behavior Trees, Blackboards, and StateTrees.
- **Audio Sounds:** Exports MetaSound graphs and SoundCue structures.
- **Data Assets:** Serializes Primary Data Assets, Data Tables, and String Tables.
- **Input Mappings:** Maps all Enhanced Input Contexts and Actions.
- **Physics:** Extracts Physical Materials and Props.
- **Project Audits:** Generates comprehensive reports on Architectural Health and Technical Debt by scanning Blueprints and C++ source code for TODO, FIXME, BUG, and HACK comments, alongside identifying orphan assets and hard references.

Analytics & Structural Health Dashboard

Once an export is complete, the 'FContextExtractionWorker' compiles disparate Analytics_*.json files into a master ledger, accessible via an interactive Slate dashboard (SExportDashboardModal).

- **Module Breakdown:** Features a custom-rendered SPieChartWidget to visually distribute the physical memory size and file count of processed asset types.
- **Blueprint Node Complexity (SummBPComp):** Tracks total nodes extracted, maps Execution Links, calculates Hard Cast Dependencies, and generates a Spaghetti Index Score to measure visual scripting complexity.
- **Architecture Health (SummArchHlt):** Evaluates Data-Driven Rate, Max Inheritance Depth, and identifies performance bottlenecks by listing Tick-Enabled Actors.
- **Technical Debt & Orphans:** Scans the code and Blueprints for pending TODO, FIXME, BUG, and HACK comments. Flags empty or unreferenced assets.
- **Top 20 Analytics:** Automatically populates interactive lists ranking the most complex Blueprints, heaviest memory assets, and most referenced assets.

Configuration & Marketplace Compliance

All extraction behaviors, level of detail (LOD) parameters, and JSONL formatting preferences are fully integrated into the tool's main interface under the EXPORT SETTINGS panel. These settings are automatically saved and persistent across your local editor sessions.

- **Marketplace Compliance:** The plugin includes a strict boolean flag (bExcludeNoAIAssets) designed to respect Epic Games' FAB "NoAI" tags, skipping flagged assets during extraction. If users choose to disable this protection to process all assets locally, they assume full responsibility for ensuring their data

processing complies with third-party EULAs.

Technical Specifications & Support

- **Supported Platforms:** Windows 64-bit
- **Supported Engine Versions:** 5.5, 5.6, 5.7
- **Support Contact:** mhvan_mx@outlook.com



